

Number: P3862R0

Date: 2025-10-06

Audience: [Library Evolution](#)

Target: C++26

Authors: [Hana Dusíková](#) and [Peter Bindels](#)

- [The problem and its inconsistency](#)
- [Urgency](#)
 - [Will you touch .substr?](#)
- [Solutions](#)
 - [Postponement](#)
 - [Removing defaulted arguments](#)
 - [Table describing how and when will string interface look like](#)
 - [What about keeping string_view everywhere?](#)
 - [Choose your own decision](#)
 - [Worse UX](#)
- [Wording](#)
 - [Wording for postponement](#)
 - [Wording for removal of default arguments](#)
 - [Feature macro](#)

P3862R0: Postpone `basic_string::subview` and w

This paper proposes temporary removing `basic_string::subview` functions added by [P3044R1](#). The functionality is really useful and brings value, but if it's kept in C++26 before adoption of `cstring_view` (zero-terminated view) we will get a suboptimal and inconsistent API without possibility of fixing it later.

Currently (as status quo on 5th October 2025) `basic_string::subview(size_type pos = 0, size_type n = npos)` returns type `basic_string_view`. During development of `cstring_view` authors of [P3655R3](#) identified issue with this functionality and for `cstring_view` this function is split into two:

```
auto cstring_view::subview(size_type pos = 0) -> cstring_view;  
auto cstring_view::subview(size_type pos, size_type n = npos) -> string_view;
```

As you can see having two overloads with zero-terminated type allows us to keep the information about zero-termination safely within type system. And we think it should be consistent on `basic_string` too.

The problem and its inconsistency

Following table shows how the `subview` functionality will be inconsistent in C++29 with

`cstring_view` available if we don't do anything now.

	string	string_view	cstring_view
	is the type zero terminated?		
	yes	no	yes
	what's return type?		
.subview()	string_view	string_view	cstring_view
.subview(off)	string_view	string_view	cstring_view
.subview(off, len)	string_view	string_view	string_view

Urgency

Something needs to be done now, because changing return type of a function is problematic. We do recognize usability of the subview feature, but we don't think it's more important over future inconsistency.

Will you touch `.substr`?

No, it's returning value type containing the substring instead of a view. And this is also argument for the change, we added `.subview` to get view, but we couldn't change `.substr` without breaking someone's code. This is the reason why we need to act now.

Solutions

We identified two solutions:

Postponement

First is simple removing of the `basic_string::subview` from the standard (partially reverting [P3044R1](#)), and keeping the function on `basic_string_view` only.

This functionality would then be fixed to do the same as `cstring_view` within the same paper and reapplied soon in 29 cycle.

Removing defaulted arguments

If we remove default arguments on `basic_string::subview` we can keep partial functionality in C++26 and introduce new overloads returning `cstring_view` later without any breakage.

Table describing how and when will string interface look like

Table describing how and when various string interfaces look like

C++26				C++29				
status quo	postponement	no default arguments		(based on status quo)	(if we do something)	(status quo) + string_view		
string								
	available	not available	not available	returns string_view (losing zero-termination + inconsistent)	return zero-terminated view	returns string_view (losing zero-termination)		
len)				returns string_view	return string_view	returns string_view		
		not available/ available			string_view			
	available	available	available	available	available	available		
len)				cstring_view (zero-terminated view)				
				.subview()		returns zero-terminated view (inconsistent with string)	returns zero-terminated view (consistent with string)	returns string_view (losing zero-termination)
	.subview(off)							
	.subview(off, len)			returns string_view	returns string_view	returns string_view		

What about keeping string_view everywhere?

This seems as the easiest and *consistent* approach, but `.subview` will lose information about the zero termination which is really useful to keep, instead this decision would force users to write: `std::cstring_view(my_string).subview(n)` instead `my_string.subview(n)` and that's very similar to motivation why `.subview` was even added. We think it's not user friendly design.

Choose your own decision

As you can see in the table:

- if we don't do anything we will have (unfixable) inconsistency later,
- if we decide the future inconsistency is bad and we want to do something, we can do:
 - postpone `.subview` for `basic_string` for a price of slightly worse UX,
 - by keeping the member function but without default arguments, we can keep it also with a slightly worse UX.

Basically the decision is about what is worse, inconsistency we can still fix, or the user experience. We think the inconsistency is worse.

Worse UX

```
auto vw1 = my_string.subview(4); // slightly shortly, but you lost the zero-terminated information
auto vw2 = std::string_view{my_string}.subview(4); // let's wait for cstring_view and type slightly longer code
auto vw3 = my_string.subview(4, std::string::npos); // with postponement it's the same amount of characters!
```

Wording

Paper contains both wording, and we ask LEWG kindly to pick one.

Wording for postponement

27.4.3.1 General [basic.string.general]

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos)
const &;
constexpr basic_string substr(size_type pos = 0, size_type n = npos)
&&;
constexpr basic_string_view<charT, traits> subview(size_type pos = 0, size_type n = npos) const;
```

27.4.3.8 String operations [string.ops]

27.4.3.8.3 basic_string::substr [string.substr]

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos)
const &;
```

¹ *Effects:* Equivalent to: `return basic_string(*this, pos, n);`

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos)
&&;
```

2 *Effects:* Equivalent to: `return basic_string(std::move(*this), pos, n);`

```
constexpr basic_string_view<charT, traits> subview(size_type pos = 0,
size_type n = npos) const;
```

~~9 *Effects:* Equivalent to: `return basic_string_view<charT, traits>(*this).subview(pos, n);`~~

Wording for removal of default arguments

27.4.3.1

General

[basic.string.general]

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos)
const &;
constexpr basic_string substr(size_type pos = 0, size_type n = npos)
&&;
constexpr basic_string_view<charT, traits> subview(size_type pos = 0,
size_type n = npos) const;
```

27.4.3.8

String operations

[string.ops]

27.4.3.8.3

basic_string::substr

[string.substr]

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos)
const &;
```

1 *Effects:* Equivalent to: `return basic_string(*this, pos, n);`

```
constexpr basic_string substr(size_type pos = 0, size_type n = npos)
&&;
```

2 *Effects:* Equivalent to: `return basic_string(std::move(*this), pos, n);`

```
constexpr basic_string_view<charT, traits> subview(size_type pos = 0,
size_type n = npos) const;
```

3 *Effects:* Equivalent to: `return basic_string_view<charT, traits>(*this).subview(pos, n);`

Feature macro

None, this is a strict removal, we just need LEWG to tell us where.